

UNIDAD 2

Gestión Lean-Agile de Productos de Software

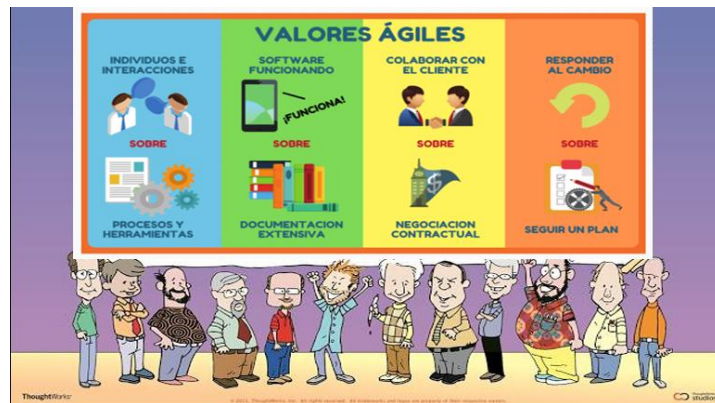
El Manifiesto Ágil

Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software. Este documento marcó un punto de inflexión al proponer un enfoque centrado en la entrega de valor, la adaptabilidad y las personas, en contraposición a los métodos tradicionales que se basaban en planes rígidos y procesos documentales pesados.

Ágil es una ideología con un conjunto definido de PRINCIPIOS que guían el desarrollo del producto.

Valores Ágiles

El Manifiesto Ágil expresa los siguientes valores. Es importante aclarar que Ágil no dice que los procesos, las herramientas, la documentación, los contratos o los planes sean inútiles o deban eliminarse. Lo que plantea es un **cambio de prioridades**.



1) Individuos e interacciones, por sobre procesos y herramientas

O en palabras más simples “Las herramientas no construyen el producto, la gente sí”.

El manifiesto plantea que el desarrollo de software es una actividad humano-intensiva. En tanto:

Se prefiere un equipo de PERSONAS MOTIVADAS con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos. Ágil lo que dice es que: Hay que dejar que quien tiene más experiencia en frontend lidere esas decisiones, sin forzar rotaciones artificiales solo por "estandarizar". También indica que las herramientas se adaptan al equipo, no al revés. Esto es clave y a veces se pierde de vista. Muchas organizaciones eligen una herramienta (por ejemplo, un framework de gestión o un software de tracking) y luego fuerzan al equipo a encajar en su forma de trabajar, incluso cuando eso genera fricción. El enfoque ágil invierte esa lógica: primero se entiende cómo el equipo trabaja mejor, y luego se buscan o configuran las herramientas que apoyen ese flujo natural, no que lo impongan.

2) Software funcionando por sobre documentación extensiva

La idea central: al priorizar el software funcionando (por sobre la documentación), el equipo entrega, al final de cada iteración, una versión del producto que:

- **Estable:** funciona correctamente, sin romper lo que ya existía.
- **Incremental:** suma funcionalidades nuevas sobre lo ya construido, en vez de entregar todo junto al final.
- **Mejorada:** cada iteración no solo agrega, sino que también pule o corrige lo anterior.

¿Para qué sirve esto? Porque al tener algo tangible y funcionando cada pocas semanas (no solo un documento), el equipo puede mostrárselo al cliente o usuario final y recibir su opinión real sobre lo que ve.

¿Por qué importa ese feedback? Porque le permite al equipo confirmar, iteración tras iteración, que está invirtiendo su tiempo en las funcionalidades que **realmente le importan al cliente**, y no en cosas que se asumieron como valiosas al principio del proyecto pero que en la práctica no cumplen sus expectativas.

Un documento de 40 páginas de requerimientos vs. un prototipo navegable: si el objetivo es que el cliente entienda cómo va a funcionar el checkout de un e-commerce, un prototipo interactivo (aunque sea con datos falsos) genera más retroalimentación real en 10 minutos de demo que semanas leyendo un documento de especificación funcional.

El enfoque ágil no elimina la documentación, sino que la hace **selectiva. Se documenta solo lo que agrega valor**. Si un documento no aporta algo útil y centrado en el cliente, no vale la pena escribirlo. Cabe aclarar si existe información necesaria de mantener ya que todo proyecto de software requiere cierto registro sobre el producto y el proceso (por ejemplo, decisiones de arquitectura, requerimientos clave, etc.). **Ágil no niega eso**, sino que la diferencia está en el "cuándo" en vez de documentar todo por adelantado "por las dudas", se documenta **cuando realmente se necesita**, en el momento en que ese documento va a ser útil. Entonces, de esto modo, la documentación se reduce a lo esencial, aunque esto no significa "cero documentaciones".

Otro aporte importante que nos enseña el enfoque ágil es que ningún documento, por bueno que sea, puede replicar la riqueza de una conversación cara a cara o de ver el software funcionando en vivo.

3) Colaboración con el cliente por sobre negociación contractual

En los enfoques tradicionales, es habitual que cliente y equipo de desarrollo firmen un **CONTRATO** al inicio del proyecto que fija **alcance, plazo y costo** (lo que se conoce como el "triángulo de hierro": scope, tiempo y presupuesto cerrados). El supuesto es que, si se negocia bien ese contrato al principio, ambas partes están protegidas. Claramente el problema es que el software se construye en un contexto de **incertidumbre y cambio constante**: el negocio evoluciona, el cliente entiende mejor su propio problema a medida que ve el producto avanzar, y el mercado cambia. Un contrato rígido no tiene forma de absorber esa realidad sin generar conflicto.

Cuando el cliente dice "me olvidé de esto" o "ahora necesito esto otro", después de haber firmado un contrato con alcance cerrado, ahí nace la **FRICCIÓN**: el equipo dice "eso no estaba en el contrato" y el cliente dice "pero lo necesito igual". Ambos tienen razón desde su posición, y esa tensión desgasta la relación. El contrato, que se firmó para dar seguridad, termina siendo un obstáculo.

¿Qué propone el enfoque Ágil?

Product Owner integrado al equipo: en Scrum, el cliente (o su representante) no espera a que termine el proyecto para opinar; participa activamente en la priorización del backlog, en el refinamiento de historias, y en cada **Sprint Review**. No es un actor externo que aprueba o rechaza al final, es parte del equipo.

Contratos ágiles o de "alcance variable": en vez de fijar 50 funcionalidades específicas por un monto cerrado, algunos equipos negocian contratos que fijan tiempo y presupuesto, pero dejan el alcance flexible ("trabajamos juntos 6 meses, priorizando lo que más valor te aporte en cada momento"). Esto permite que, si aparece una nueva prioridad, se pueda incorporar sin necesidad de renegociar todo el contrato.

Feedback en cada sprint en vez de una única aprobación final: si el cliente prueba el software cada 2 semanas y da su opinión, los cambios de rumbo se hacen a tiempo y de forma económica. Si el cliente solo ve el producto al final (como marca el contrato), cualquier cambio implica retrabajo masivo y renegociación.

Cliente presente en las pruebas: no se trata de mandarle un mail con capturas de pantalla, sino de que el cliente use el software real, lo pruebe con datos reales, y dé **feedback directo**. Eso genera compromiso genuino con el resultado, no solo una firma de aceptación.

4) Responder a Cambios por sobre Seguir un Plan

La gestión tradicional de proyectos parte de un supuesto: si al inicio se define bien el plan (requerimientos, cronograma, alcance), el trabajo consiste en **ejecutar ese plan y controlar las desviaciones**. Cualquier cambio se ve como un problema, algo que hay que evitar o, si ocurre, gestionar como una excepción costosa (change request, aprobación, renegociación).

El problema de fondo surge debido a que **es imposible que un cliente conozca con certeza absoluta todas sus necesidades al comienzo de un proyecto**.

- El propio cliente entiende mejor lo que necesita **a medida que ve el software funcionando**, no antes.
- El mercado, la competencia o la regulación pueden cambiar durante el desarrollo.

En ágil, el plan **no desaparece**, pero deja de ser un documento fijo escrito al principio: se convierte en algo vivo, que se ajusta constantemente en base a lo que se va aprendiendo y ese conocimiento no aparece solo al principio del proyecto, sino que se va generando **iteración tras iteración**.

Este valor **no significa "no planificar"**, así como el valor anterior no significaba "no documentar". Ágil sigue planificando, pero lo hace de forma **incremental y adaptativa**: se planifica con el nivel de detalle adecuado para el corto plazo, y con mayor flexibilidad para el largo plazo.

Principios del Manifiesto Ágil



1. Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor

Explicación fundamentada:

Esto se soporta mediante el uso de *User Stories* (Historias de Usuario). Una Historia de Usuario obliga al equipo a programar "porciones verticales" del sistema que el usuario pueda utilizar (interfaz, lógica y base de datos de una funcionalidad específica), en lugar de programar tareas técnicas horizontales (como "crear todas las tablas de la base de datos", lo cual no aporta valor directo e inmediato al cliente). Al entregar las Historias de Usuario de mayor prioridad primero, el cliente puede empezar a utilizar el software y obtener beneficios económicos mucho antes de que el sistema completo esté terminado.

Ejemplo Práctico:

- En el desarrollo de una tienda virtual, en lugar de que los programadores dediquen un mes entero a configurar la arquitectura de TODA la base de datos (una tarea técnica sin valor visible para el usuario), el equipo toma la Historia de Usuario de mayor prioridad: "Como comprador, quiero pagar con tarjeta de crédito para recibir mi pedido en casa". En una iteración, el equipo programa la interfaz de pago, la lógica de validación y las tablas de base de datos **SOLO** para esa funcionalidad. Al finalizar, el cliente ya tiene una tienda capaz de procesar ventas reales y generar ingresos, satisfaciendo su necesidad principal de forma temprana.

2. Aceptar que los requisitos cambian, incluso en etapas tardías del desarrollo

La ingeniería de software tradicional asume que el costo de realizar un cambio crece exponencialmente a medida que avanza el proyecto, por lo que exige "congelar" los requerimientos al inicio. La bibliografía Lean y Ágil demuestra que esto genera un gran desperdicio, ya que obliga a tomar decisiones cuando se tiene la menor cantidad de información disponible.

Lo que lo distingue de la gestión tradicional es el matiz "**incluso en etapas tardías**". No alcanza con estar abierto al cambio al principio del proyecto (eso cualquier metodología lo acepta); la diferencia real está en aceptar el cambio cuando ya se avanzó bastante, sin tratarlo como una crisis o una falla del proceso.

Ejemplo concreto: si en la iteración 8 de 12 el cliente pide cambiar la lógica de cálculo de precios porque cambió una regulación impositiva, un equipo tradicional (con alcance cerrado por contrato) lo trataría como un cambio costoso y conflictivo. Un equipo ágil lo incorpora al backlog, lo prioriza según su urgencia real, y lo desarrolla en las próximas iteraciones, entendiendo que ese cambio es tan legítimo como cualquier requerimiento original.

Esto es posible justamente porque, como vimos antes, no hay un plan rígido de 6 meses firmado en piedra: hay un backlog vivo que se reordena constantemente.

3. Entregar software funcional frecuentemente (entre dos semanas y dos meses, con preferencia al periodo más corto)

En el desarrollo de software, el trabajo parcialmente terminado (código escrito, pero no probado, o probado, pero no integrado) es un desperdicio que oculta errores graves. Agile soluciona esto dividiendo el trabajo en ciclos de tiempo fijos y cortos llamados iteraciones o *Sprints* (generalmente de 1 a 4 semanas).

Al final de cada iteración el equipo no puede entregar código "al 90%". El software debe cumplir rigurosamente con la **Definición de Hecho** (*Definition of Done - DoD*), lo que significa que el código debe estar escrito, refactorizado, probado (pruebas unitarias y de integración pasadas) y sin errores conocidos. El resultado debe ser un **incremento potencialmente entregable** (*Potentially Shippable Product Increment*); es decir, una pieza de software con una calidad tan alta que el cliente podría decidir ponerlo en los servidores de producción ese mismo día si así lo deseara.

FILOSOFÍA ÁGIL

¿Por qué existe el enfoque ágil? y qué lo diferencia estructuralmente de otros modelos iterativos. Vamos por partes.

El objetivo primordial: entregar software frecuente en un entorno cambiante

¿por qué existe ágil?

El objetivo primordial: entregar software frecuente en un entorno cambiante

Ágil surge como respuesta a proyectos donde los requerimientos no se pueden fijar con certeza al principio, por dos razones concretas:

1. El cliente no puede predecir el impacto real del sistema hasta que lo usa en su operación diaria.
2. El entorno de negocio cambia durante el desarrollo (mercado, competencia, regulaciones), y eso puede volver obsoleto algo que tenía sentido al comienzo del proyecto.

¿cómo responde ágil a ese problema?

El ciclo iterativo-incremental

- **Iterativo:** el trabajo se hace en ciclos repetidos, no de una sola vez.
- **Incremental:** cada ciclo suma una porción nueva y funcional al sistema ya construido.

El concepto clave que sostiene ese ciclo: el Empirismo

El conocimiento se construye a partir de la **experiencia y observación real**, no de la planificación teórica anticipada.

El equipo no asume que sabe de antemano cómo va a resultar el sistema completo; lo **descubre iteración a iteración**, observando resultados reales.

Por qué esto distingue a ágil de otros modelos iterativos (PUD/RUP)

Este es el punto más sutil: **PUD/RUP también son iterativos-incrementales**, pero son **procesos definidos**, no empíricos.

	Proceso Definido (PUD/RUP)	Proceso Empírico (Ágil)
Supuesto de base	Se puede predecir el resultado siguiendo el proceso correctamente	Hay demasiada incertidumbre para predecir; hay que descubrir sobre la marcha
Estructura	Iterativo, pero con fases y artefactos bastante definidos de antemano	Iterativo, pero flexible en su ejecución

Es decir: ser iterativo **no alcanza** para ser ágil. Lo que define a ágil es aceptar que **ni siquiera el proceso mismo puede especificarse del todo de antemano**, porque la incertidumbre es demasiado alta.

Dos consecuencias prácticas de este mecanismo

a) El cliente inmerso en el proceso

Los incrementos (típicamente cada 2-3 semanas) se ponen a disposición del cliente para obtener feedback real. **El cliente no espera pasivamente al final; participa activamente inspeccionando cada incremento y ayudando a decidir el rumbo del siguiente.** *[Conecta con el valor 3 del manifiesto: colaboración con el cliente.]*

b) Comunicación informal en lugar de documentación formal

No es solo "documentar menos": es reemplazar documentación por comunicación frecuente e informal (daily standups, charlas, mensajes rápidos) **en vez de reuniones formales con actas y documentos firmados.** *[Conecta con el valor 2 del manifiesto: software funcionando sobre documentación extensiva.]*

¿qué tan estructurado debe ser ágil?

La definición de Martin Fowler

"Un compromiso entre nada de proceso y demasiado proceso."

Extremo	Qué pasa	Resultado
Demasiado proceso (tradicional)	El proceso se vuelve un fin en sí mismo: importa cumplir el checklist, llenar el documento, seguir la ceremonia	El proceso se antepone al resultado
Nada de proceso	Programar sin disciplina, sin planificación ni revisión	Caos, falta de previsibilidad, calidad inconsistente
Ágil (punto medio)	Suficiente estructura para ser predecible y sostenible (sprints, ceremonias, roles), sin caer en rigidez burocrática	El proceso sirve al equipo y al resultado, no al revés

La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales, la transparencia, la inspección y la adaptación.

Los 3 pilares del empirismo

Este cierre es clave porque son los pilares que sostienen técnicamente todo lo anterior (son, de hecho, los mismos pilares que después formaliza Scrum):

1. **Transparencia:** todos los aspectos relevantes del proceso deben ser visibles para quienes son responsables del resultado. Ejemplo: un backlog visible para todo el equipo y el cliente, no información oculta o repartida en silos.
2. **Inspección:** se debe revisar frecuentemente el progreso hacia el objetivo, para detectar variaciones o problemas. Ejemplo: la Sprint Review, donde se inspecciona el incremento construido.
3. **Adaptación:** si la inspección revela que algo se está desviando de lo aceptable o de lo que el cliente necesita, hay que ajustar el proceso o el producto lo antes posible. Ejemplo: la Retrospectiva, donde el equipo ajusta su forma de trabajar en base a lo aprendido.

Sin estos tres pilares funcionando juntos, el ciclo iterativo-incremental pierde sentido: se podría iterar sin aprender nada, si no hay transparencia real o si nadie inspecciona y adapta en base a lo que se observa.

¿Cuándo es aplicable Ágil?

Este texto responde una pregunta distinta a las anteriores: no "qué es ágil" sino "en qué contextos tiene sentido usarlo". Es un tema clave porque ágil no es una solución universal para cualquier tipo de proyecto.

Ágil no es un framework, es una ideología

Este primer punto es fundamental y muchas veces se confunde en la práctica:

- **Ágil** = un conjunto de valores y principios (el Manifiesto). Define un *para qué* y un *cómo pensar*, no un *cómo hacer* paso a paso.
- **Los frameworks ágiles** (Scrum, Kanban, XP, SAgile, etc.) = implementaciones concretas de esa ideología. Son los que sí definen roles, ceremonias, artefactos específicos.

Ejemplo: decir "somos ágiles" no dice mucho por sí solo. Decir "usamos Scrum con sprints de 2 semanas" ya es una implementación concreta de esa ideología ágil.

Un matiz importante: gestión vs. ingeniería

La mayoría de las prácticas ágiles son de gestión de proyectos, no de ingeniería de software. Scrum, por ejemplo, te dice cómo organizar el trabajo del equipo (sprints, backlog, roles), pero no te dice cómo programar bien.

El modelo de certeza/entendimiento: ¿dónde funciona mejor ágil?

Esta es la parte central del texto. Se basa en un modelo (conocido como la Matriz de Stacey o modelos similares) que clasifica los problemas

Zona	Características	Enfoque recomendado
Simple	Alta certeza y alto entendimiento. Se sabe qué hay que hacer y cómo hacerlo.	Procesos predictivos/tradicionales funcionan bien
Complicado	Requiere expertise, pero es resoluble con análisis	Procesos más estructurados, con expertos
Complejo	Entendimiento y certeza parciales. Se sabe algo, pero hay que ir descubriendo el resto sobre la marcha	Acá es donde ágil da mejores resultados
Caótico / "Anarquía"	Gran incertidumbre, casi no hay entendimiento previo	Investigación, experimentación exploratoria (no ágil "puro" de entrega)

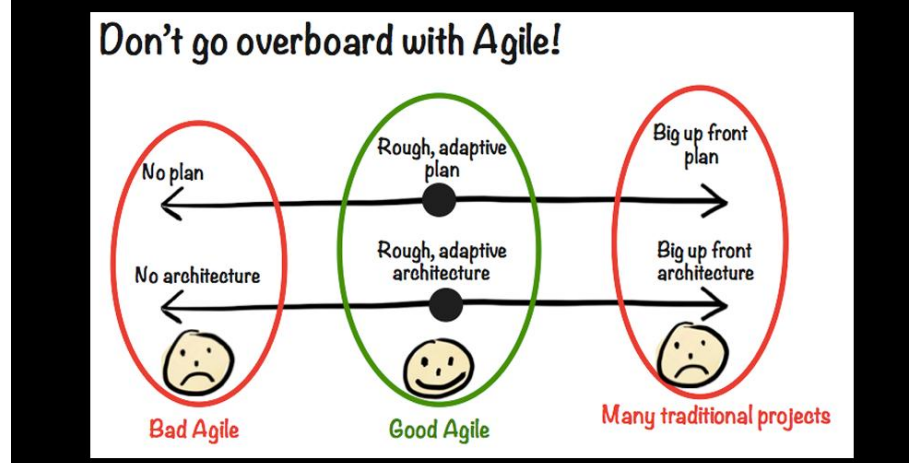
Por qué ágil brilla específicamente en "lo complejo"

Un **producto nuevo** es el ejemplo típico de este cuadrante: hay una idea de negocio, hay cierto entendimiento del problema, pero no hay certeza total sobre qué funcionalidades exactas se van a necesitar ni cómo van a reaccionar los usuarios. Es demasiada incertidumbre para planificar todo de antemano (por eso no aplica bien un enfoque predictivo tradicional), pero hay suficiente entendimiento como para poder iterar y aprender con cada incremento (por eso no es pura "anarquía" tampoco).

"Las metodologías ágiles son aplicables en el contexto de lo complejo (dan buenos resultados), en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial."

Ejemplo: desarrollar una nueva app de fintech para un mercado que recién se está formando. No hay certeza absoluta sobre qué funcionalidades van a adoptar los usuarios, pero tampoco se está totalmente a ciegas: hay hipótesis de negocio razonables que se pueden ir validando iteración a iteración. Ese es el terreno ideal para ágil.

Ser Ágil no es ser indisciplinado.



"Ser ágil no es ser indisciplinado"

Mucha gente interpreta "ágil" como sinónimo de "trabajar sin estructura, improvisando sobre la marcha, sin compromiso ni disciplina". Es exactamente lo contrario de lo que plantea la filosofía ágil.

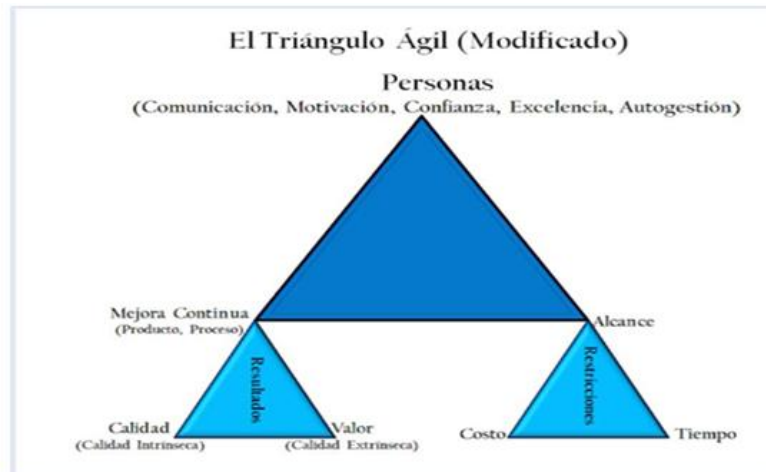
Como vimos antes con la cita de Fowler ("un compromiso entre nada de proceso y demasiado proceso"), ágil **requiere** disciplina: cumplir con la Definition of Done, respetar los tiempos de las ceremonias, mantener la transparencia del backlog, testear antes de dar algo por terminado. La flexibilidad está en el *qué* se construye y en *qué orden* (adaptación continua), no en la *falta de rigor* con la que se trabaja.

¿Por qué ir a Ágil?

- **Acelera entregas de valor:** al trabajar en iteraciones cortas, el cliente recibe funcionalidades usables cada pocas semanas, no al final del proyecto.
- **Adaptación a cambios:** el backlog se Re prioriza constantemente, así que un cambio de requerimiento se absorbe en la próxima iteración, sin romper todo el plan.
- **Alineación entre negocio y TI:** al tener al cliente/PO priorizando según valor real de negocio, el equipo técnico no trabaja en cosas desconectadas de los objetivos de la empresa.
- **Reduce riesgos:** entregar seguido con feedback temprano permite detectar errores (técnicos o de negocio) cuando todavía son baratos de corregir, no al final del proyecto.
- **Mejor visibilidad:** gracias a la transparencia (backlog visible, demos frecuentes), todos saben el estado real del proyecto, sin depender de reportes desactualizados.

- **Mejora relaciones de equipo:** la comunicación fluida y la colaboración constante (en vez de silos) fortalecen la confianza entre el equipo y con el cliente.
- **Reduce costos:** al detectar errores o cambios de rumbo temprano, se evita el retrabajo masivo de corregir algo cuando ya está todo construido sobre una base equivocada.

El Triángulo Ágil



El triángulo ágil apunta a un producto de software que pone a las **personas** (que tienen ciertas características como confianza, motivación, etc.) en el lugar más importante ya que las personas son las que hacen el software. Donde se prioriza que haya comunicación, motivación, confianza, excelencia y autogestión.

Debajo encontramos el pie, en el cual tenemos la **mejora continua** (tanto del **producto** (vía iteraciones) como del **proceso** (vía comunicación del equipo)). Es el triángulo de resultados, teniendo como pie a la calidad la cual es calidad intrínseca, las características técnicas que el equipo le da al producto (buen código, bien construido), y la calidad extrínseca, el impacto real que el producto tiene para el cliente.

Del otro lado de la base podemos encontrar el **alcance**, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.

Finalmente se tienen en cuenta las variables que se usan para gestionar el trabajo (alcance- costo-tiempo).

Debemos hacer foco en entregarle al cliente un producto de calidad que se vaya mejorando y evolucionando en forma continua.

Claramente cambia la pregunta central del proyecto: en vez de "¿cumplimos alcance, costo y tiempo?" (triángulo de hierro tradicional), pregunta "**¿esto genera calidad y valor real para el cliente?**"

Alcance, costo y tiempo **siguen existiendo**, pero pasan a ser **restricciones**, no el objetivo.

Requerimientos en ambientes ágiles

Lo Más Difícil del Software: Decidir Qué Construir

La idea central es simple: el mayor riesgo de un proyecto de software no es técnico, es de comunicación.

El verdadero motivo de fracaso de los proyectos

La afirmación central es fuerte: la tecnología rara vez es el obstáculo real. Hoy existen lenguajes, frameworks y herramientas más que suficientes para resolver casi cualquier problema técnico. Lo que falla, una y otra vez, es que el equipo construye algo distinto de lo que el cliente realmente necesitaba.

Esto se conoce históricamente como la "crisis del software": un término que surge ya desde fines de los años 60, cuando se empezó a notar que los proyectos de software sistemáticamente se entregaban tarde, se pasaban de presupuesto, o simplemente no cumplían lo que el usuario esperaba, a pesar de contar con buenos programadores y buena tecnología disponible.

Por qué se subestima el relevamiento de requerimientos

Relevar requerimientos es una actividad social, no técnica. Esto es fácil de pasar por alto porque, a simple vista, "juntar requerimientos" parece un paso administrativo, casi burocrático, antes de "empezar a trabajar de verdad" (que sería programar).

Pero en realidad, entender qué necesita un cliente implica:

- Interpretar necesidades que el propio cliente muchas veces no sabe verbalizar con precisión.
- Traducir jerga de negocio a especificaciones que tengan sentido técnico.
- Detectar contradicciones o ambigüedades en lo que se pide.

Ejemplo: un cliente dice "necesito un reporte de ventas". Detrás de esa frase simple hay decenas de preguntas sin responder: ¿ventas por qué período? ¿Discriminadas por vendedor, por producto, por región? ¿Se necesita en tiempo real o basta con actualización diaria? Si el equipo asume respuestas en lugar de conversarlas, es muy probable que el reporte final no sirva, aunque esté perfectamente programado.

El problema de comunicación: los tres escenarios posibles

Esta es la parte más rica del texto, porque plantea que el desequilibrio de poder en la comunicación, en cualquier dirección, lleva al fracaso:

Escenario 1: Domina el cliente/negocio

El cliente impone funcionalidades y plazos sin que el equipo técnico pueda dar su feedback real sobre viabilidad.

Ejemplo: el área comercial le promete a un cliente externo una nueva funcionalidad "para el mes que viene" sin haber consultado al equipo de desarrollo, que en realidad necesita el doble de tiempo por complejidad técnica no contemplada. El equipo termina bajo presión, entregando algo apurado y de baja calidad, o incumpliendo el plazo de todas formas.

Escenario 2: Domina el equipo técnico

Los desarrolladores hablan en términos técnicos, imponen soluciones basadas en lo que a ellos les resulta interesante o elegante de construir, y dejan de escuchar activamente al cliente.

Ejemplo: un equipo decide migrar a una arquitectura de microservicios "porque es mejor práctica", sin que eso responda a una necesidad real del negocio en ese momento, mientras el cliente sigue esperando funcionalidades que le importan y no llegan.

Escenario 3 (el óptimo): Comunicación equilibrada entre ambas partes

Ninguna de las dos partes "gana" la conversación; ambas negocian y se escuchan. El cliente entiende las restricciones técnicas y de tiempo; el equipo entiende el valor de negocio detrás de cada pedido.

Esto conecta directamente con el **valor 3 del manifiesto** (colaboración con el cliente) y con el **valor 1** (individuos e interacciones): la comunicación fluida y bidireccional no es un "extra", es la condición necesaria para que el proyecto tenga sentido.

Por qué las entregas tempranas ("early versions") son tan valiosas

Acá se conecta con todo lo visto sobre entregar software funcionando frecuentemente. La idea es simple pero potente: **es mucho más barato descubrir un malentendido temprano que descubrirlo al final.**

Cuando el cliente ve una versión temprana e incompleta del producto:

- Puede darse cuenta de que "no era exactamente esto lo que quería", a tiempo de corregir el rumbo.
- Puede tener ideas nuevas que no había pensado al principio, precisamente porque recién al ver algo tangible logra imaginar mejor lo que necesita.

Ejemplo: un cliente pide una app de gestión de turnos. Recién al ver la primera versión funcionando (aunque sea con pocas funcionalidades), se da cuenta de que en realidad necesita que los turnos se puedan reprogramar fácilmente, algo que no había mencionado porque no lo había pensado hasta verlo en uso.

Por qué no es posible un PERT perfecto

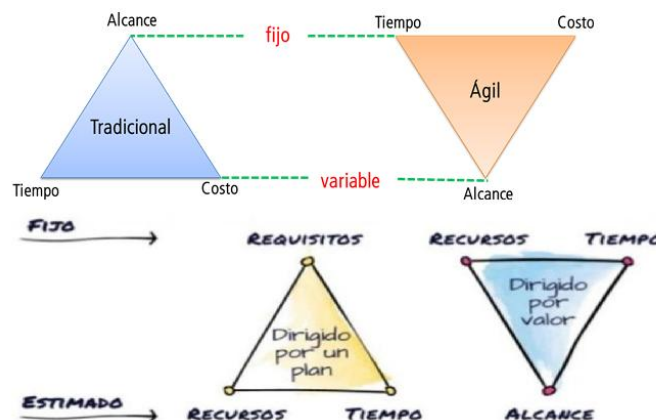
El diagrama PERT (una herramienta clásica de gestión de proyectos que mapea todas las actividades, dependencias y tiempos de un proyecto de punta a punta) asume que **se puede conocer de antemano todo el trabajo a realizar.** No se puede planificar todo de antemano

El texto conecta esta idea con todo lo anterior: **si ni siquiera el cliente sabe con certeza qué necesita hasta que ve el producto avanzar, es lógicamente imposible planificar con precisión absoluta todas las actividades desde el día uno.** Cualquier PERT "completo" de un proyecto de software sería, en el mejor de los casos, una aproximación optimista; en el peor, una ficción que genera una falsa sensación de control.

Esto retoma directamente la idea de **procesos empíricos vs. procesos definidos** que vimos antes: como hay incertidumbre genuina sobre el "qué", no tiene sentido fingir certeza total sobre el "cuándo" y el "cómo" desde el principio.

El software no falla por falta de buena tecnología, falla por falta de buena comunicación entre cliente y equipo — por eso ágil prioriza el diálogo constante y las entregas tempranas, en vez de intentar "adivinarlo todo" al principio.

La Triple Restricción: Enfoque Ágil vs. Tradicional



Este texto retoma la idea del "triángulo de hierro" que mencionamos antes, pero ahora profundiza en **qué variable se fija y cuál se deja flexible** en cada enfoque. Es una de las diferencias más prácticas y concretas entre ágil y la gestión tradicional.

La triple restricción: alcance, tiempo y costo

Todo proyecto se mueve dentro de estas tres variables, que están interrelacionadas: si tocas una, necesariamente afectas a las otras dos. La pregunta clave es: **¿cuál de las tres se fija primero, y cuáles se derivan de esa?**

Enfoque tradicional: se fija el alcance

En la gestión tradicional, la secuencia lógica es:

1. **Primero se define el alcance:** se relevan todos los requerimientos, se especifica con detalle qué funcionalidades va a tener el producto.
2. **A partir de ese alcance, se derivan tiempo y costo:** una vez que se sabe "qué" hay que construir, se estima cuánto va a tardar y cuánto va a costar.

Por qué el cliente prefiere esta lógica: el alcance es lo que representa **lo que el cliente realmente necesita** — las funcionalidades concretas que resuelven su problema. Es lógico que quiera tener control sobre eso primero, y recién después pregunte "¿cuánto tiempo y cuánto dinero me va a llevar tener todo esto?".

El problema que esto genera: si más adelante el cliente pide un cambio de requerimientos (agregar una funcionalidad, modificar una ya definida), ese cambio impacta directamente sobre un alcance que ya estaba "cerrado" y sobre el cual ya se habían calculado tiempo y costo. Por eso cualquier cambio se vuelve costoso y conflictivo: hay que recalcular todo lo que se había derivado de ese alcance original, y eso genera fricción (como vimos antes con el ejemplo del "me olvidé de esto").

Ejemplo: un contrato tradicional que establece "se entregarán 50 funcionalidades específicas, en 8 meses, por \$100.000". Si a mitad de camino el cliente pide una funcionalidad nueva, hay que renegociar plazo y/o costo, generando la clásica discusión de "eso no estaba en el alcance original".

Enfoque ágil: se fijan tiempo y recursos

Ágil invierte completamente esta lógica, partiendo de una premisa realista: **el alcance no se puede conocer con certeza al principio**, porque los requerimientos van a cambiar si o sí a medida que el proyecto avanza (como vimos en textos anteriores).

Entonces, en vez de intentar fijar algo que por naturaleza es incierto (el alcance), ágil fija lo que sí se puede controlar con precisión:

1. **Se fija el tiempo:** por ejemplo, sprints de duración constante (2 semanas), o una fecha de release definida.
2. **Se fijan los recursos:** el equipo de desarrollo tiene un tamaño y una capacidad conocida (no se contratan ni despiden personas cada vez que cambia un requerimiento).
3. **El alcance se convierte en la variable que se ajusta:** dado ese tiempo fijo y ese equipo fijo, la pregunta pasa a ser "¿cuánto software podemos entregar con estas condiciones?", y ese "cuánto" se ajusta constantemente según lo que se prioriza en cada iteración.

Por qué esto es más realista: la capacidad de un equipo (cuántas personas son, qué tan rápido pueden trabajar de forma sostenible) es algo relativamente estable y medible con el tiempo (por ejemplo, a través de la velocidad del equipo en Scrum). En cambio, el alcance completo de un producto es, por naturaleza, algo que se va descubriendo. Fijar lo estable y dejar flexible lo incierto es más coherente que al revés.

Ejemplo: un equipo ágil trabaja con sprints de 2 semanas y un equipo fijo de 5 personas. En cada sprint, se pregunta "¿qué es lo de mayor valor que podemos construir con esta capacidad, en este tiempo?", en vez de comprometerse de antemano a que en 8 meses van a estar las 50 funcionalidades exactas que se imaginaron al principio.

Por qué esto reduce el costo de los cambios

En el enfoque tradicional, cambiar un requerimiento significa tocar la variable que estaba fija (el alcance), lo cual obliga a recalcular las variables derivadas (tiempo y costo) — de ahí la fricción y el conflicto. En **ágil**, en cambio, el alcance **ya es la variable flexible por diseño**. Cuando aparece un nuevo requerimiento o cambia una prioridad, no hace falta renegociar tiempo ni costo: simplemente se reordena qué entra en el alcance de la próxima iteración, dentro de la misma capacidad de equipo y el mismo ritmo de entregas que ya estaban establecidos.

BRUF – Big Requirements Up Front



La imagen adjuntada expresa la frecuencia con la que los usuarios utilizan determinadas funcionalidades de un software.

A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos. Además, es prácticamente imposible conocer al 100% todas las necesidades de los usuarios, ya que ni ellos saben concretamente qué necesitan, ni cómo explicar de forma abarcadora y completa sus necesidades. Por otro lado, esas necesidades que tienen los usuarios al momento de ser relevadas, con el tiempo probablemente irán cambiando, con lo cual, la priorización es un factor fundamental en esa etapa, donde el equipo en conjunto con el Product Owner, deberán ser capaces de decidir cuáles son aquellos requerimientos indispensables para el negocio, de manera que, si se implementan esas necesidades, se puede entregar una nueva versión del producto que sea de valor para el cliente.

BRUF es la práctica tradicional de intentar **relevar y documentar el 100% de los requerimientos antes de construir nada**. Es lo opuesto al enfoque ágil.

Por qué falla, en 3 ideas

1. **Los datos lo desmienten:** un pequeño grupo de funcionalidades concentra la mayor parte del uso real; el resto casi no se usa. Si invertís meses documentando todo, gran parte de ese esfuerzo termina en cosas que nadie usa.
2. **Es imposible conocer el 100% desde el inicio:** ni el propio usuario sabe con precisión qué necesita, ni podría explicarlo completo y sin ambigüedad, aunque quisiera.
3. **Las necesidades cambian con el tiempo:** lo que se relevó al principio probablemente ya no sea lo mismo que se necesita después.

La consecuencia práctica

En vez de relevar todo de una vez, hay que **priorizar constantemente**: el equipo y el Product Owner identifican qué es lo indispensable *ahora* para el negocio, lo construyen, y entregan valor real sin esperar a tener el alcance completo definido.

En una frase: no tiene sentido documentar todo por adelantado, porque buena parte de eso no se va a usar, no se puede conocer con certeza, y de todas formas va a cambiar — mejor priorizar seguido y entregar lo que realmente importa en cada momento.

Product Backlog y Requerimientos Just in Time

Estos dos conceptos son la traducción operativa de todo lo que venimos viendo: si BRUF fracasa porque intenta relevarlo todo por adelantado, el **Product Backlog** y los **requerimientos Just in Time** son las herramientas concretas que ágil propone en su lugar.

Product Backlog

Qué es y quién lo define

El Product Backlog es, en esencia, una **lista viva y priorizada** de todo lo que el producto podría necesitar: **funcionalidades, mejoras, correcciones**. La palabra clave es "priorizada": no es una lista plana de pedidos, sino un orden que refleja qué es más importante construir primero.

Lo crucial acá es **quién decide ese orden**: el **Product Owner**, porque es quien tiene mayor claridad sobre **las necesidades reales del negocio y del cliente**. No es el equipo técnico quien decide qué se construye primero según lo que le resulte más cómodo o interesante de programar.

La comparación con el enfoque tradicional

Este es el punto más interesante del texto, porque señala una paradoja del enfoque tradicional: **se supone que el cliente tiene el control del alcance (como vimos en la triple restricción), pero en la práctica, muchas veces el cliente tradicional no se compromete activamente** con la priorización del día a día, y termina delegando esa tarea al equipo de desarrollo.

El problema de esto: el equipo técnico, por más buena voluntad que tenga, no tiene el mismo conocimiento del negocio que el cliente. Si es el equipo quien decide qué construir primero, es muy probable que priorice según criterios técnicos (lo más fácil, lo más interesante, lo que encaja mejor en la arquitectura) en lugar de criterios de valor de negocio real. El resultado: un producto que cumple con el alcance pactado, pero que **no necesariamente satisface al cliente**.

Ejemplo: en un proyecto tradicional, si el cliente entrega un documento de 100 requerimientos y luego desaparece hasta la entrega final, el equipo técnico va a decidir el orden de construcción según su propio criterio. Puede terminar construyendo primero un módulo de reportes avanzados (técnicamente interesante) mientras posterga una funcionalidad core que el cliente necesitaba con urgencia, simplemente porque nadie priorizó activamente desde el lado del negocio.

En ágil, en cambio, es el **Product Owner (representante directo del cliente/negocio)** quien prioriza activamente y de forma continua, asegurando que lo que se construye primero sea realmente lo más valioso.

Cómo esto compensa la falta de documentación formal

Esta es una idea que conecta varios hilos que venimos viendo: si no hay un documento de requerimientos exhaustivo y cerrado desde el principio (como vimos en el valor 2), ¿qué garantiza que no se pierda información importante?

La respuesta ágil es: **un cliente presente y comprometido, más comunicación cara a cara constante, reemplazan la función que cumplía ese documento**. En lugar de que la "verdad" del proyecto esté encerrada en un papel que se redactó una sola vez, la verdad se construye y actualiza permanentemente a través del diálogo directo entre el Product Owner y el equipo. Esto es más rico y más preciso que cualquier documento estático, como vimos antes.

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



Requerimientos Just in Time

La gestión Ágil de requerimientos sostiene los “Requerimientos justo a tiempo” asociados a los principios de eliminar el desperdicio y definir compromisos de la filosofía Lean.

- **Eliminar el desperdicio (waste):** no invertir esfuerzo en cosas que no van a generar valor. Releva y documentar requerimientos que después van a cambiar, o que nunca se van a usar (como vimos con BRUF), **es exactamente ese tipo de desperdicio**.
- **Diferir compromisos:** no tomar decisiones definitivas antes de que sea estrictamente necesario. Cuanto más se demora una decisión (sin bloquear el avance), más información real se tiene disponible para tomarla bien.

Cómo funciona en la práctica

El proceso arranca con algo liviano: **una visión general del producto y algunos requerimientos identificados** (no todos, solo los suficientes para empezar a construir con sentido, como vimos en el pilar de “sólo lo suficiente”). A partir de ahí:

1. Se empieza a construir con esa base inicial.
2. Se incorpora al cliente activamente para que dé feedback sobre lo que se va construyendo.
3. Ese feedback genera nuevo conocimiento, que a su vez define los próximos requerimientos a trabajar.

Es decir, los requerimientos **no se relevan todos de una vez**, sino que se van “descubriendo y detallando” justo en el momento en que hacen falta, ni antes ni después. Por eso “just in time”.

Ejemplo: en vez de especificar con lujo de detalle, desde el día uno, cómo va a funcionar el módulo de reportes de un sistema (que quizás se va a construir recién en el mes 4), el equipo lo deja como una idea general en el backlog. Recién cuando se acerca el momento de trabajar en esa funcionalidad, se hace el relevamiento detallado, con la información más actualizada posible sobre lo que el negocio necesita en ese momento (que puede ser distinta a la que había en el mes 1).

El Product Backlog nunca está al 100%

Este es el punto que cierra el concepto, contrastando directamente con el “Up Front Specification” del enfoque tradicional.

El enfoque tradicional: todo especificado antes de empezar

Bajo Up Front Specification, se define el 100% de los requerimientos antes de escribir una sola línea de código. El riesgo de esto es que **cualquier error de interpretación en esa etapa inicial se arrastra durante todo el proyecto**, y se descubre recién al final, cuando ya es carísimo corregirlo.

Ejemplo: si en el relevamiento inicial se malinterpretó cómo debía funcionar un flujo de aprobación de pagos, y eso no se descubre hasta la entrega final (meses después), corregirlo implica rehacer buena parte del sistema, con el costo y el retraso que eso conlleva.

El enfoque ágil: consenso continuo, menor impacto de los errores

Como en ágil los requerimientos se van consensuando constantemente con el cliente (a través del Product Backlog, priorizado y refinado iteración a iteración), **un error de interpretación se detecta mucho antes, típicamente en la revisión del incremento donde ese requerimiento se implementó**. El costo de corregirlo es mucho menor porque no se arrastró durante meses.

Por eso se dice que **el Product Backlog nunca está “completo” al 100%**: siempre es un documento vivo, en construcción, que se sigue refinando y agregando ítems a medida que el proyecto avanza y se aprende más. No es una falla del proceso que esté “incompleto”; es, de hecho, la característica que lo hace funcionar bien.

En síntesis: el Product Backlog traslada la priorización al cliente (a través del Product Owner), **en lugar de dejarla en manos del equipo técnico como pasaba en el enfoque tradicional**, lo que asegura que se construya lo que realmente aporta valor. Y los requerimientos Just in Time evitan el desperdicio de especificar todo por adelantado, descubriendo cada detalle justo cuando hace falta, lo que reduce drásticamente el impacto de cualquier error de interpretación.

Fuente de requerimientos ¿De dónde salen?

Enfoque tradicional: la fuente es un documento formal, la **ERS (Especificación de Requerimientos de Software)**. El proceso es: alguien releva los requerimientos, los escribe en ese documento, y el cliente después lo **lee y valida** (por eso el texto lo asocia a "papel" en el gráfico). La comunicación pasa fundamentalmente por escrito.

Enfoque ágil: la fuente es la **conversación cara a cara** con el cliente. No hay un documento intermedio que capture "la verdad" del requerimiento; la verdad se construye en el diálogo directo.

Por qué el cara a cara es superior para elicitar requerimientos

La **elicitación de requerimientos no es solo escuchar palabras**, es un proceso donde importa también **cómo se dicen las cosas** — el lenguaje corporal, los gestos, las dudas o titubeos del cliente al explicar algo. Un documento escrito pierde toda esa capa de información.

Ejemplo: si le preguntás a un cliente "¿necesitás que el reporte se pueda exportar a Excel?", y responde "sí, claro, eso estaría bien" con un tono dudoso y sin mucho entusiasmo, un analista atento puede detectar que en realidad no es tan prioritario como parece en el papel. Esa señal se pierde completamente si el mismo requerimiento aparece como un ítem más en una lista de un documento ERS.

!!!**IMPORTATE!!!**

ágil no significa "no documentar ni especificar nada". Significa que se prioriza documentar **lo que realmente tiene valor**, en lugar de invertir el esfuerzo en construir una ERS completa que, de todas formas, probablemente contenga errores (por las razones que ya vimos: ni el cliente puede anticipar todo, ni transmitir todo con precisión en un documento estático).

Momento de la captura: ¿cuándo se relevan?

Enfoque ágil: los requerimientos se relevan **de forma continua, durante todo el proyecto**, no en una etapa separada. El proyecto arranca sin que los requerimientos estén completamente definidos — solo con una **visión general** — y esa definición se va completando iteración a iteración, a medida que se construye y se aprende.

Contenedor y persistencia: ¿dónde "viven" los requerimientos?

Enfoque ágil: el **Product Backlog es un contenedor transitorio**, no permanente. Esto es clave: un ítem del backlog no está pensado para "vivir para siempre" ahí. Una vez que ese requerimiento se implementa, **deja de necesitar existir como ítem de backlog, porque ya se "persiste" de otra forma: en el propio software funcionando**.

Es decir, la pregunta "¿qué hace el sistema?" ya no se responde leyendo un documento, se responde **usando el producto real**. El software mismo se convierte en la documentación viva de lo que existe.

¿Y el detalle fino de por qué se construyó así? Los casos de prueba

Aunque el ítem del backlog "desaparece" una vez implementado, el **detalle de valor y comportamiento esperado no se pierde**, queda registrado en los **casos de prueba (testing)** que se construyeron para validar esa funcionalidad.

Tipos de Requerimientos

La pirámide de requerimientos: dos dominios distintos

La clasificación divide los requerimientos en dos grandes dominios, según a quién le importan:

Dominio del Problema (arriba de la pirámide) — le importa al cliente

- **Requerimientos de Negocio:** son el nivel más alto. Representan el **objetivo de fondo**, el "para qué" del proyecto en términos de negocio. Por ejemplo: "aumentar la retención de clientes" o "reducir el tiempo de atención en soporte".
- **Requerimientos de Usuario:** son un nivel más concreto que el de negocio, pero siguen estando del lado de "el problema", no de "la solución técnica". Describen lo que el usuario necesita hacer con el sistema para que ese objetivo de negocio se cumpla. Por ejemplo: "el usuario necesita poder resolver sus dudas sin esperar a un agente humano".

Dominio de la Solución (abajo de la pirámide) — no le interesa al cliente

- **Requerimientos de la Solución/técnicos:** son el "cómo" se traduce todo eso en código ejecutable. Al cliente, en principio, no le importa si se usó tal base de datos, tal arquitectura, o tal patrón de diseño; le importa que su problema se resuelva. Esto queda del lado del equipo técnico.



Ejemplo aplicado a los tres niveles:

- **Negocio:** "Reducir el abandono de carritos de compra."
- **Usuario:** "Como comprador, quiero guardar mi carrito para completarlo después, para no perder lo que ya elegí."
- **Solución/técnico:** implementar una tabla en base de datos para persistir carritos, un job que los limpie después de X días, etc. (esto es indiferente para el cliente).

User Stories: requerimientos de usuario alineados a valor de negocio

Este es el punto más importante para retener: **las User Stories no son requerimientos de negocio ni requerimientos técnicos, son requerimientos de usuario, pero siempre pensados como el medio para lograr un requerimiento de negocio**

Principios ágiles relacionados con requerimientos ágiles



1- LA PRIORIDAD ES SATISFACER AL CLIENTE A TRAVÉS DE RELEASES TEMPRANOS Y FRECUENTES (2 SEMANAS A UN MES)



2- RECIBIR CAMBIOS DE REQUERIMIENTOS, AUN EN ETAPAS FINALES



4- TÉCNICOS Y NO TÉCNICOS TRABAJANDO JUNTOS TODO EL PROYECTO



6 - EL MEDIO DE COMUNICACIÓN POR EXCELENCIA ES CARA A CARA



11 - LAS MEJORES ARQUITECTURAS, DISEÑOS Y REQUERIMIENTOS EMERGEN DE EQUIPOS AUTOORGANIZADOS

Principio 1: La prioridad es satisfacer al cliente

El Product Owner ordena el backlog según valor de negocio, y el equipo entrega incrementos seguidos (cada 2-4 semanas) para que el cliente reciba valor real cuanto antes, en lugar de esperar meses por una entrega única. Esto conecta directo con lo que acabamos de ver de BRUF: entregar seguido permite validar y ajustar los requerimientos con la realidad, en vez de intentar adivinarlos todos de una vez.

Principio 2: Recibir cambios de requerimientos, aun en etapas finales

También lo vimos: **acepta que los requerimientos van a cambiar incluso avanzado el proyecto, y lo trata como algo normal (no como una crisis), gracias a que no hay un plan rígido cerrado desde el principio.**

Principio 4: Técnicos y no técnicos trabajando juntos todo el proyecto

Qué dice: el negocio (PO, cliente) y el equipo técnico no deben trabajar en compartimentos separados, con el negocio "entregando requerimientos" y el equipo "programando en otra habitación". Deben trabajar **codo a codo, durante todo el proyecto**, no solo al principio.

User Stories

La idea más importante: "un mecanismo para diferir una conversación"

La intuición equivocada es pensar que una User Story es como un mini-documento de requerimientos: se escribe, contiene toda la información necesaria, y con eso alcanza para programar. **Esto es falso**. La User Story es apenas **un recordatorio escrito de que hay que tener una conversación** sobre esa necesidad, en el momento en que se vaya a trabajar en ella.

Conecta directamente con el concepto de requerimientos Just in Time que vimos antes. En vez de tener esa conversación detallada ahora (cuando hay menos información e incertidumbre alta), **se posterga la conversación profunda hasta el momento en que realmente se va a construir esa funcionalidad**, cuando hay más contexto disponible.

Una User Story **no especifica comportamiento del sistema a nivel técnico**, se queda en el nivel de lo que el usuario necesita. Es voluntariamente poco detallada; el detalle llega después, en la conversación y en los criterios de aceptación.

Item de planificación, no documento de por vida

Esto retoma lo que vimos con el Product Backlog como "contenedor transitorio": **la User Story tiene una vida útil limitada, sirve para planificar el trabajo**, pero una vez implementada, no queda como referencia permanente del sistema. Para eso —para documentar cómo usar el sistema ya construido— existe el **manual de usuario**, que es un artefacto distinto con otro propósito.

Escritas por el Product Owner

Las User Stories representan necesidades de usuario conectadas a valor de negocio, tiene sentido que quien las redacte sea quien tiene esa visión combinada — el PO.

"Verticales"

Este es un punto que vale la pena explicar bien porque no es intuitivo. **Decir que una User Story es vertical significa que atraviesa todas las capas técnicas necesarias** para entregar valor de punta a punta, en lugar de limitarse a una sola capa.

Para entender la diferencia, pensemos en el opuesto: una historia "horizontal" sería algo como "diseñar la interfaz de usuario del checkout" (solo toca la capa de frontend) o "crear las tablas de base de datos para pedidos" (solo toca la capa de persistencia). Ninguna de estas, por sí sola, le entrega valor real al usuario: una interfaz sin lógica de negocio detrás no sirve de nada, y una tabla de base de datos sin nada que la use tampoco.

Una historia vertical, en cambio, es algo como "Como cliente quiero pagar mi pedido con tarjeta de crédito, para completar mi compra". Para que esto funcione de punta a punta, **necesita: una pantalla (frontend), lógica de validación (backend), una tabla donde persistir el pago (base de datos), y quizás una integración con una pasarela de pago (servicio externo)**. Todo eso junto, aunque toque varias capas técnicas, constituye **una sola unidad de valor completo** para el usuario.

Por qué esto importa: si las historias se dividieran por capa técnica (horizontal), ninguna entrega valor por sí sola hasta que estén todas las capas terminadas, lo cual va en contra de la idea de entregar incrementos funcionales y completos en cada iteración (Definition of Done, valor 2 del manifiesto). Por eso se busca que las historias sean verticales.

Las 5 funciones "multipropósito" de las User Stories

1. **Representan una necesidad de usuario, sin mucho detalle:** como ya vimos, es apenas el disparador de la conversación, no la especificación completa.
2. **Conforman una descripción del producto:** el conjunto de historias del backlog, en su totalidad, da una idea general de qué hace o va a hacer el producto (aunque de forma incompleta y cambiante).
3. **Son ítems de planificación:** se estiman, se priorizan, se organizan en sprints. Son la unidad básica de trabajo que el equipo planifica.
4. **Recordatorios de futuras conversaciones:** ya lo vimos, es la función central.
5. **Junto con el código, documentan el producto:** acá se retoma algo que vimos antes — el software funcionando (y sus tests) es, en definitiva, la documentación más confiable de lo que el sistema realmente hace. La historia por sí sola no documenta nada preciso; combinada con el código que efectivamente se construyó a partir de ella, sí aporta ese contexto histórico de "por qué se construyó así".

Las 3 Partes de la User Story: Tarjeta, Conversación y Confirmación

1. Tarjeta (Card)

Es la parte física/escrita, la más visible, pero —como ya vimos— **es apenas un recordatorio**, no la **especificación completa**.

La sintaxis: 4 componentes clave

"Como [rol], yo puedo [actividad], de forma tal que [valor de negocio]"

Nombre del rol: quién ejecuta la acción o recibe el beneficio. No alcanza con decir "usuario" en general; **cuanto más específico el rol, mejor se entiende el contexto**. *Ejemplo:* no es lo mismo "como cliente" que "como cliente frecuente con más de 10 compras" — el segundo puede justificar una funcionalidad distinta (por ejemplo, un descuento por fidelidad).

Actividad: la acción concreta que el sistema debe permitir. Es el "qué". *Ejemplo:* "puedo filtrar mis pedidos por rango de fechas".

Valor de negocio: el texto lo remarca como **la parte más importante**, y tiene sentido — es el "por qué". Sin esto, la historia se convierte en una tarea aislada sin justificación, exactamente el problema que vimos al hablar del primer valor del manifiesto (evitar tareas que no aporten valor). *Ejemplo:* "...para poder encontrar rápidamente una compra específica sin tener que revisar todo el historial." Este componente es también el que justifica que el **Product Owner sea quien escriba las historias**: solo alguien con visión de negocio puede argumentar con solidez el "para qué", y ese "para qué" es lo que permite **priorizar** el backlog con criterio real (recordemos: valor de negocio = base de la priorización).

Frase verbal (opcional): un nombre corto para referirse a la historia rápidamente en conversaciones o en el backlog, sin tener que repetir la oración completa cada vez. *Ejemplo:* en vez de decir "la historia de 'como cliente quiero filtrar mis pedidos por fecha para encontrar una compra rápido'", el equipo simplemente **dice "la historia de filtro de pedidos"**.

Criterios de aceptación

Son condiciones que delimitan qué incluye y qué no incluye la historia, dando al equipo una visión más clara de su alcance, sin caer todavía en detalles de implementación (se mantienen a **alto nivel de abstracción**).

Ejemplo para la historia de filtrar pedidos por fecha:

- **El filtro debe permitir seleccionar una fecha de inicio y una de fin.**
- **Si no hay pedidos en el rango seleccionado, debe mostrarse un mensaje claro** (sin especificar todavía el texto exacto ni el diseño).
- **El filtro debe combinarse con otros filtros ya existentes** (por ejemplo, por estado del pedido).

Nota que esto ya da más estructura que la frase original de la historia, pero **todavía no dice cómo se implementa técnicamente** (no dice si es un date picker, un input de texto, qué librería se usa, etc.) — eso queda para la implementación real.

Pruebas de aceptación

Acá sí se llega a un nivel de detalle mucho más operativo, pensado para que desarrolladores y testers sepan exactamente cuándo algo "está bien hecho".

Ejemplo, siguiendo con la misma historia:

- Dado que ingreso una fecha de inicio posterior a la fecha de fin, entonces el sistema debe mostrar un error de validación.
- Dado que selecciono un rango de fechas sin pedidos, entonces debe mostrarse el mensaje "No se encontraron pedidos en este período".
- Dado que combino el filtro de fecha con el filtro de estado "cancelado", entonces solo deben mostrarse los pedidos cancelados dentro de ese rango.

Estas pruebas **pueden agregarse o quitarse en cualquier momento**, porque a medida que avanza la conversación (o incluso durante el desarrollo), **pueden surgir nuevos casos borde que nadie había pensado al principio**. Esto es coherente con todo lo que venimos viendo sobre incertidumbre y descubrimiento progresivo.

2. Conversación

Esta es la **parte más importante** de toda la User Story, aunque sea la menos "visible" o tangible de las tres. Es el momento donde efectivamente se resuelve todo lo que la tarjeta, a propósito, dejó sin especificar.

La conexión con dos ideas del manifiesto

- **"Acuerdo por sobre negociación del contrato"** (valor 3): en vez de que la tarjeta funcione como una cláusula contractual rígida ("esto es lo que se pidió, ni más ni menos"), la conversación permite llegar a un entendimiento compartido y flexible.
- **"Técnicos y no técnicos trabajando juntos"** (uno de los 12 principios que probablemente veamos más adelante): **la conversación es, literalmente, el Product Owner (representante del negocio) y el equipo técnico sentados juntos, resolviendo ambigüedades en tiempo real.**

El rol del Product Owner en esta conversación

El PO es parte del equipo, no un cliente externo que aprueba desde lejos. Por eso se espera de él **responsabilidad, compromiso y confianza** — **no puede desentenderse de la conversación y dejar que el equipo "adivine" los detalles**; tiene que estar genuinamente presente y comprometido para llegar a acuerdos reales.

¿La conversación se pierde? No necesariamente

Aunque la conversación en sí es efímera (una charla no queda escrita automáticamente), el texto aclara que:

- **Se puede dejar registro en grabaciones, minutas, o herramientas de gestión.**
- Pero, sobre todo, **su contenido se termina plasmando de forma distribuida** en la tarjeta (criterios de aceptación) y en las pruebas de aceptación. No hace falta un acta formal de la conversación; sus conclusiones más importantes quedan capturadas ahí.

3. Confirmación

Es el acuerdo explícito entre PO y equipo sobre **cuándo una historia se considera realmente terminada**. Se apoya directamente en las pruebas de aceptación definidas anteriormente: si el sistema pasa esas pruebas, la historia está confirmada como completa.

El detalle clave: es el **Product Owner quien decide si las pruebas de aceptación son válidas** — es decir, la validación final de "esto realmente cumple lo que se necesitaba" no queda en manos del equipo técnico exclusivamente, sino que requiere la conformidad de quien representa el valor de negocio.

El ciclo completo, con un ejemplo integrado

1. **Tarjeta:** "Como cliente frecuente, quiero recibir un descuento automático en mi décima compra, para sentirme recompensado por mi fidelidad." + criterios de aceptación de alto nivel (ej. "el descuento debe aplicarse automáticamente al llegar a la compra número 10").
2. **Conversación:** el equipo y el PO se sientan a discutir: ¿el descuento es un porcentaje fijo o varía? ¿Se acumulan compras de todos los canales o solo online? ¿Qué pasa si el cliente devuelve una de las 9 compras anteriores, sigue contando? De ahí salen las pruebas de aceptación detalladas.
3. **Confirmación:** una vez implementado, el equipo corre las pruebas de aceptación acordadas, y el PO valida que efectivamente el comportamiento resultante es el que se había consensuado. Si todo pasa, la historia se da por terminada (Done).

Ventajas

Prioriza la comunicación verbal por sobre la escrita

La historia por sí sola no alcanza; lo importante pasa en **la Conversación** (una de las 3 C) entre PO y equipo. Esto evita el problema clásico de los documentos extensos, que se vuelven ambiguos o quedan desactualizados apenas algo cambia.

Comprensibles tanto para el cliente como para el equipo técnico

No hace falta jerga técnica para entenderlas. Esto evita el problema de comunicación que vimos antes (cuando domina la jerga técnica y el negocio no entiende, o al revés). *Ejemplo:* "Como cliente quiero recibir un email al confirmar mi compra" lo entiende tanto el dueño del negocio como el desarrollador, sin necesidad de hablar de colas de mensajes ni APIs.

Tamaño justo para planificar

Las historias están pensadas para ser lo suficientemente chicas como para completarse dentro de una iteración (sprint), pero lo suficientemente grandes como para representar una unidad de valor real (recordemos: son verticales). Ese tamaño intermedio es justo el que permite **estimar, organizar y comprometerse** con un plan de trabajo de corto plazo, sin caer ni en tareas microscópicas sin sentido de negocio, ni en épicas gigantes imposibles de planificar con precisión.

Permiten el desarrollo iterativo

Al dividir todo el producto en unidades chicas de valor (historias), el equipo puede ir completando y entregando una historia por vez, generando incrementos funcionales seguidos, en vez de necesitar terminar "todo el sistema" de una sola vez.

Permiten desestimar detalles hasta entender mejor la necesidad

Esta ventaja retoma directamente la idea de "diferir la conversación" y los requerimientos Just in Time: **no** hace falta resolver todo el detalle al momento de escribir la historia; se puede dejar para más adelante, cuando haya más claridad real sobre lo que el usuario necesita.

Desventajas

En proyectos grandes, es difícil ver la relación entre historias.

Pero en proyectos grandes, con cientos de historias acumuladas en el backlog, se pierde fácilmente la **visión de conjunto**: cómo se relacionan entre sí, qué dependencias ocultas existen, cómo encajan dentro de un flujo más amplio del producto.

No son apropiadas para equipos demasiado grandes

Como la ventaja central de las User Stories depende de la **conversación verbal directa** entre PO y equipo, esto funciona bien cuando el equipo es chico y todos pueden estar involucrados en esas conversaciones de forma fluida. Cuando el equipo crece mucho, se vuelve logísticamente difícil sostener esa comunicación cara a cara con la misma calidad: hay más personas, más sub-equipos, más dificultad para que todos estén alineados solo con conversación informal.

Ejemplo: en un equipo de 6 personas, el PO puede conversar directamente con todos sobre cada historia sin mayor esfuerzo. En una organización con 15 equipos y 100+ desarrolladores trabajando sobre el mismo producto, ya no alcanza con "conversación verbal": hace falta mayor coordinación formal, más documentación compartida, y frameworks de escalado (como SAFe o LeSS) que agregan estructura adicional, precisamente porque las User Stories solas no escalan bien a ese tamaño.

Product Backlog

Estructura del Product Backlog: una pila priorizada

El Product Backlog es una **pila de PBIs** (Product Backlog Items — las User Stories son el tipo más común, pero no el único posible). La disposición de esa pila no es arbitraria:

- **Arriba:** historias de **mayor prioridad y granularidad fina** (chicas, bien detalladas, listas para trabajarse pronto).
- **Abajo:** historias de **menor prioridad y granularidad gruesa** (grandes, poco detalladas, porque todavía no vale la pena invertir tiempo en refinarlas).

Cómo se prioriza y quién lo hace

La priorización **no es estática**: **se puede reordenar en cualquier momento, y quien tiene esa potestad es siempre el Product Owner, en base al valor de negocio de cada ítem** (como ya vimos en textos anteriores).

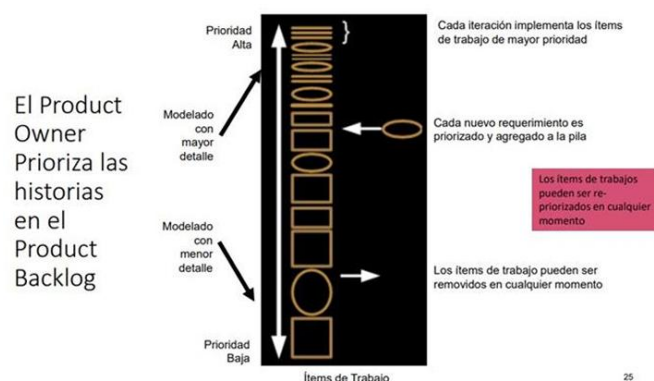
Cómo entran y salen ítems del backlog

Entrada de nuevos ítems: **en cualquier momento del proyecto pueden surgir nuevas necesidades (por feedback de usuarios, cambios de negocio, nuevas ideas), y se van agregando al backlog**. Esto es consistente con la idea de que el backlog nunca está "cerrado".

Salida de ítems (para implementarlos): acá aparece un concepto clave — la **velocidad del equipo** (medida en Story Points por iteración). Al planificar una nueva iteración, el equipo mira cuántos Story Points suele completar históricamente por sprint (su velocidad), y en base a eso decide cuántas historias de las que están arriba de la pila puede comprometerse a sacar del backlog para esa iteración.

Por qué el backlog nunca llega al 100%

Esta conclusión ya la habíamos adelantado en textos anteriores, pero acá se justifica con el mecanismo concreto: **como constantemente entran ítems nuevos (por descubrimiento continuo) y salen ítems (por implementación), el backlog está en permanente movimiento. Nunca hay un momento donde se pueda decir "esto es el 100% definitivo del producto", porque es, por diseño, un artefacto vivo y en construcción continua.**



Porciones verticales: repaso rápido con foco en arquitectura

Ya vimos este concepto con las User Stories, pero acá se lo conecta explícitamente con la **arquitectura en capas** de un sistema típico:

- **Capa de presentación** (interfaz de usuario)
- **Capa de lógica de negocio** (reglas, validaciones, procesamiento)
- **Capa de datos** (persistencia, base de datos)

Una historia vertical necesita "un poquito de cada capa" para poder entregar una funcionalidad completa y utilizable. Esto es lo opuesto a dividir el trabajo por capa (lo cual, como vimos, generaría historias horizontales que no entregan valor por sí solas).

Tamaño

La recomendación general es clara: **muchas historias chicas, mejor que pocas historias grandes**. Esto facilita la planificación, permite entregas más frecuentes, y reduce el riesgo de que una historia grande esconda complejidad no descubierta.

El tamaño se mide en **Story Points**, una unidad relativa (no de tiempo directo) que combina tres factores:

1. Complejidad

Se refiere a una dificultad **intrínseca de la funcionalidad**, que no se puede simplemente resolver dividiendo la historia en partes más chicas.

Ejemplo: un algoritmo de recomendación de productos basado en el historial de compras de un usuario tiene complejidad inherente (lógica de cálculo, múltiples variables a considerar), independientemente de cuánto se intente trocear la historia.

2. Esfuerzo

Es más simple: el tiempo estimado, en horas de trabajo lineales / horas hombre, que va a llevar implementarlo. Distinto de la complejidad, porque algo puede ser simple conceptualmente, pero llevar mucho tiempo (por ejemplo, cargar manualmente muchos datos de configuración), o complejo pero rápido de programar una vez que se entiende bien.

3. Incertidumbre

Este factor tiene dos variantes:

- **Incertidumbre técnica:** dudas sobre el "cómo" construirlo, típicamente por falta de experiencia del equipo con cierta tecnología. *Ejemplo:* el equipo nunca trabajó con reconocimiento facial y debe implementar una funcionalidad que lo requiere; hay incertidumbre sobre cuánto tiempo y esfuerzo real va a llevar, más allá de la complejidad "de libro" del problema.
- **Incertidumbre de negocio:** dudas sobre cómo el usuario realmente va a interactuar con la funcionalidad, generalmente por falta de conocimiento profundo del dominio. *Ejemplo:* un equipo que nunca trabajó en el rubro de seguros tiene que implementar el cálculo de una póliza, y no tiene claridad sobre todas las reglas de negocio reales que aplica una aseguradora en ese cálculo, más allá de lo que le explicaron superficialmente.

Story Points

Son unidades de medida relativas (no importa el valor en sí, sino su comparación), las cuales **miden el tamaño de un producto, NO de una medida basada en el tiempo** (por ejemplo, horas hombre).

Niveles de abstracción

Esta parte resuelve una pregunta lógica: si el backlog tiene historias muy detalladas arriba (listas para trabajar) y cosas más vagas abajo (como vimos en el texto anterior), **¿cómo se le llama a esas cosas vagas de abajo, y en qué se diferencian formalmente de una User Story?**

1. **User Story:** la unidad "lista para trabajar"

El criterio que la define en este nivel es concreto: **cumple con el DoR (Definition of Ready)**, es decir, está lo suficientemente refinada, clara y de tamaño adecuado como para poder entrar a una iteración y completarse dentro de ella.

2. **Épica:** una User Story "demasiado grande"

Esta es la definición más interesante del texto, porque **el criterio para distinguir una épica de una User Story no es un tamaño absoluto fijo**, sino algo puramente relativo al contexto del equipo: **si entra en una iteración o no**.

- Si una historia se puede empezar y terminar dentro de una sola iteración → es una User Story.
- Si es demasiado grande para completarse en una sola iteración → es una **Épica**.

Esto es coherente con la idea de Story Points relativos: no hay un número mágico universal de puntos que separe una historia de una épica; depende de la capacidad del equipo y la duración de sus iteraciones.

Otra señal para detectar una épica: cuando una historia no cumple con el criterio de calidad **INVEST**, eso suele ser indicio de que en realidad es una épica disfrazada de historia, y necesita dividirse.

3. **Tema:** un agrupador conceptual, no un tamaño

Son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas., que se agrupan para poder tratarlas como una sola unidad a la hora de planificar o estimar a alto nivel.

Ejemplo: un tema podría ser "Checkout y pagos", que agrupa historias como "agregar tarjeta de crédito", "aplicar cupón de descuento", "seleccionar método de envío", "confirmar pedido". Cada una de esas historias puede tener tamaños distintos (algunas podrían incluso ser épicas todavía sin dividir), pero todas están agrupadas bajo el mismo tema porque conceptualmente pertenecen al mismo flujo de trabajo del producto.

Criterios de Aceptación

Es una nota que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que este debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Lo fundamental es que esté escrito en términos objetivos y concretos.

Las 5 funciones que cumplen

1. **Definen límites de la historia:** qué entra y qué no entra dentro del alcance de esa historia puntual.
2. **Ayudan al PO a clarificar sus propios requerimientos mínimos:** a veces el ejercicio de escribir el criterio obliga al propio PO a pensar con más precisión qué es lo que realmente necesita.
3. **Generan visión compartida** entre todo el equipo sobre qué se está construyendo.
4. **Sirven de base para derivar pruebas** (tanto de desarrolladores como de testers).
5. **Marcan el límite de "hasta dónde" desarrollar:** evitan el problema de que un desarrollador siga agregando funcionalidad "porque parece una buena idea", sin que eso estuviera realmente pedido (scope creep dentro de una misma historia).

¿Dónde van los detalles finos, entonces?

Esta es una pregunta lógica que el texto responde directamente: si el criterio de aceptación se mantiene a alto nivel, ¿dónde se registran esos detalles concretos que sí importan (nombre exacto de una columna, formato de un número, tipo de control de interfaz)?

Ejemplos de esos detalles (tal como los da el texto):

- El encabezado de una columna debe decir "Saldo".
- El formato del saldo es "999.999.999,99".
- Se debe usar una lista desplegable en vez de un checkbox.

Estos detalles, que surgen naturalmente de la **conversación** entre PO y equipo (retomando las "3 C"), se guardan en uno de dos lugares, según decida cada equipo:

1. **Documentación interna** del equipo (wikis, notas técnicas, etc.).
2. **Pruebas de aceptación automatizadas** (donde el detalle queda codificado directamente como parte de un test).

No hay una regla única impuesta; **cada equipo define su propia convención** sobre dónde y cómo mantener ese nivel de detalle.

Pruebas de aceptación

Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación. El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen. Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; suelen usarse para completar detalles de la US. Es importante mencionar que las mismas deben escribirse antes que la programación empiece, y que se recomienda que las escriba el cliente.

DoR (Definition of Ready)

Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

I N V E S T - MODEL

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

I — Independiente

Una historia debe poder implementarse en cualquier orden, sin depender de que otra historia se haga antes.

N — Negociable

La historia no es un contrato cerrado, es un recordatorio para conversar. Este criterio remarca algo clave: la historia debe centrarse en el "qué", nunca en el "cómo".

V — Valuable (Valiosa)

Una US debe ser valorable para los clientes o usuarios (el para qué de la descripción). Es decir, las U.S. deben aportar valor de negocio a quiénes estará destinado el producto, no para el equipo de desarrollo.

E — Estimable

La US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo.

Las 3 razones por las que una historia puede no ser estimable, y su solución:

1. **Incertidumbre de dominio (de negocio):** el equipo no entiende bien la necesidad real. → Solución: **conversar más con el PO** para aclarar dudas antes de estimar.
2. **Incertidumbre técnica:** el equipo no sabe cómo se resolvería técnicamente (por ejemplo, tecnología desconocida). → Solución: hacer un **Spike** — una investigación técnica acotada en tiempo, cuyo objetivo es reducir esa incertidumbre (no construir la funcionalidad final, sino explorar y aprender lo suficiente para poder estimar después con más confianza).
3. **La historia es demasiado grande:** es en realidad una épica. → Solución: **dividirla en historias más chicas**, cada una estimable por separado.

S — Small (Pequeña)

La US debe ser lo suficientemente pequeña como para ser finalizada en una iteración. Se debe comenzar y terminar una característica en una iteración, no debe haber trabajos a medias. El tamaño es relativo, ya que depende del equipo, de la experiencia que tiene, de su capacidad de trabajo y también depende de la duración de la iteración. Ágil aplica gestión binaria, es decir, si se puede completar la US en la iteración, es pequeña.

T — Testable

Una historia es **Testable** cuando se puede definir, de forma **objetiva y verificable**, si quedó bien implementada o no. Es decir: tiene que haber una forma concreta de decir "sí, esto cumple" o "no, esto no cumple" — sin que dependa de la opinión subjetiva de cada persona.

Estimación en Ambientes Ágiles

Este texto aborda un tema clave que atraviesa toda la gestión de proyectos: **qué es estimar, por qué es tan difícil, y por qué no hay que confundirlo con planificar o comprometerse**. Vamos a profundizarlo.

Qué es estimar

La definición de McConnell que cita el texto es clara: **estimar es predecir cuánto tiempo o costo va a llevar un proyecto** (o, en nuestro contexto, una historia, un sprint, un release). **Pero el matiz más importante viene después: estimar no es planificar ni comprometerse**.

La distinción clave: estimación ≠ plan ≠ compromiso

Esta es, probablemente, la idea más importante de todo el texto, y conviene separarla bien en tres conceptos distintos:

- **Estimación:** una aproximación, basada en la información disponible en ese momento. Es inherentemente incierta.
- **Plan:** se construye a partir de la estimación, pero incorpora otras variables (prioridades, recursos disponibles, restricciones del negocio). El plan final puede terminar siendo distinto de lo que la estimación pura sugería.
- **Compromiso:** una promesa concreta hacia otra persona o parte interesada. Una estimación **no** es una promesa.

Ejemplo para diferenciarlos: el equipo estima que una historia tiene 8 Story Points (estimación). El PO decide que, dado el equipo y la velocidad histórica, esa historia entrará en el sprint 5 (plan). El equipo se compromete con el PO a entregarla completa al final de ese sprint (compromiso). Son tres cosas relacionadas, pero conceptualmente distintas — y confundirlas es una fuente común de conflictos (por ejemplo, cuando alguien trata una estimación inicial como si fuera una promesa firme).

Errores en las estimaciones

¿Por qué estimar es tan difícil?

1. **Información imprecisa** sobre el software a construir o sobre la capacidad real del equipo. Si no se sabe bien qué hay que hacer, o no se conoce bien qué tan rápido puede trabajar el equipo, cualquier número que se dé va a estar mal.
2. **Proyecto mal definido / caótico:** si ni siquiera el alcance general está claro, estimar en detalle es prácticamente adivinar.
3. **Imprecisión propia del proceso de estimación:** incluso con buena información, el método usado para estimar puede introducir error (por ejemplo, técnicas mal aplicadas, falta de consenso en el equipo).
4. **Omitir actividades necesarias:** esta es muy común en la práctica. Se estima solo el "trabajo de programar", pero se olvidan cosas como reuniones, revisiones de código, gestión de licencias, testing, documentación mínima, etc. Todo eso consume tiempo real que no se contempló.
5. **Confundir estimar con planificar:** ya lo vimos, son cosas distintas.
6. **A mayor brecha entre lo planeado y lo estimado, mayor el riesgo:** El plan no tiene por qué coincidir con la estimación (ya lo vimos), pero este punto agrega algo importante: **cuanto más se aleje el plan de lo que la estimación real sugería, más riesgo corre el proyecto**.
7. **Tratar las estimaciones como compromisos:** Una **estimación** es una aproximación hecha con la información disponible en ese momento (con margen de error, como vimos: puede fallar hasta 4x al principio del proyecto). Un **compromiso** es una promesa concreta hacia otra persona. Son cosas distintas, pero en la práctica se mezclan todo el tiempo.

¿Por qué estimamos?

El texto es claro en que no se estima "porque sí" o solo por costumbre burocrática, sino por dos motivos concretos:

1. **Predecir completitud:** tener una noción razonable de cuándo algo podría estar listo, para poder coordinarse con otras áreas, comunicar expectativas, planificar releases.
2. **Administrar riesgos:** cuanto mejor se entiende el tamaño y la incertidumbre de un trabajo, mejor se pueden anticipar problemas y tomar decisiones a tiempo (por ejemplo, decidir si conviene hacer un Spike antes de comprometerse, como vimos con INVEST).

Qué requiere estimar bien

Tres ingredientes, según el texto:

- **Experiencia:** haber vivido situaciones similares ayuda a calibrar mejor el juicio.
- **Buena información histórica (métricas):** por ejemplo, la velocidad real del equipo en sprints anteriores, que es mucho más confiable que una estimación teórica sin datos de respaldo.
- **El valor de comprometerse con números cuando solo hay información cualitativa:** A veces hay que dar una estimación, aunque la información disponible sea imprecisa o cualitativa ("creemos que es complejo, pero no sabemos exactamente cuánto"). **Hace falta cierta valentía profesional para dar un número aproximado en esas condiciones, en lugar de negarse a estimar.**

Por qué las primeras estimaciones son tan poco confiables: el "error del 400%"

Este es un dato muy citado en la literatura de estimación de software, y el texto lo explica bien:

- **Al principio del proyecto hay poca información y mucha incertidumbre:** todavía no se conoce bien el alcance, la tecnología, ni el propio equipo trabajando junto. Es lógico que cualquier estimación en ese momento tenga un margen de error enorme.
- **Las estimaciones tienden a ser optimistas por naturaleza:** hay un sesgo humano bien documentado (a veces llamado "falacia de planificación") donde las personas subestiman sistemáticamente cuánto va a llevar algo, incluso cuando ya tuvieron experiencias pasadas de subestimación.

Por eso se dice que la primera estimación puede diferir hasta en un **factor de 4x (400%)** respecto al resultado real — y por eso mismo, **las estimaciones deben recalcularse constantemente** a medida que el proyecto avanza y hay más información disponible (esto conecta directamente con todo lo visto sobre empirismo, inspección y adaptación).

Proceso de Estimación

<>

Métodos Utilizados para Estimar

<>

Problemas de estimaciones tradicionales

<>

ESTIMACIONES EN AMBIENTES AGILES

Quién estima: el equipo, no el Product Owner

La filosofía ágil parte de un principio claro: **quien hace el trabajo es quien debe estimarlo. Por eso es el equipo de desarrollo, de forma colectiva, quien estima — y el Product Owner queda explícitamente afuera de ese proceso.**

Por qué la estimación colectiva funciona mejor que la individual

En ágil, al estimar en grupo:

- Se generan **debates** que sacan a la luz puntos de vista o riesgos que una sola persona no habría detectado.
- Se necesita una **actitud de escucha real**: todos deben poder opinar libremente, sin que una voz dominante (por ejemplo, la del desarrollador más senior) "tape" la opinión de los demás.

Ejemplo: si un desarrollador junior cree que una historia es más compleja de lo que parece porque conoce un detalle técnico puntual (por ejemplo, una limitación de una librería), y nadie le da espacio para decirlo, esa información valiosa se pierde y la estimación grupal termina siendo peor que si se hubiera escuchado a todos por igual.

El problema del miedo: subestimar vs. sobreestimar

Este es un punto muy interesante y realista sobre la psicología detrás de estimar, especialmente cuando hay temor a la reacción del cliente:

Subestimar (por miedo al rechazo)

El equipo, temiendo que un número alto genere enojo o desconfianza del cliente, **da un número más bajo del real**, sin contemplar todo el trabajo verdaderamente necesario.

Ejemplo: el equipo sabe que una funcionalidad probablemente lleve 3 sprints completos, pero por miedo a que el cliente diga "eso es demasiado tiempo", estima 2 sprints, ocultando parte del trabajo real necesario (como testing exhaustivo o casos borde).

Sobreestimar (por cubrirse ante imprevistos)

El caso opuesto: se infla el número "por las dudas", generando una especie de colchón de seguridad personal.

Ejemplo del propio texto: alguien sabe que una tarea le va a llevar 2 horas, pero dice 4, "por si pasa algo en el medio". Esto genera una estimación artificialmente alejada de la realidad.

Por qué ambos comportamientos son problemáticos

Los dos casos comparten el mismo problema de fondo: **la estimación deja de reflejar la realidad del trabajo y empieza a reflejar el miedo o la ansiedad de quien estima.** Esto rompe la utilidad misma de estimar (recordemos: predecir completitud y administrar riesgos), **porque el número resultante ya no es información confiable, es una reacción emocional disfrazada de dato.**

La solución de fondo pasa por construir un ambiente de **confianza** entre equipo, PO y cliente (retomando el valor 1 del manifiesto), donde estimar con honestidad no se sienta amenazante.

El beneficio real de estimar: el proceso en sí, más que el número

Esta es una idea que vale la pena destacar: el documento afirma que **el gran beneficio de estimar es el propio acto de hacerlo**, no tanto el número final obtenido.

¿Por qué? Porque al sentarse a estimar en grupo, el equipo:

- Se ve obligado a **analizar la historia con más profundidad** de lo que haría si nadie estimara.
- **Enriquece su comprensión colectiva** del trabajo, lo cual mejora también futuras estimaciones (se genera aprendizaje acumulado, calibración de criterio).
- Detecta **tempranamente si algo es factible o no**, incluso antes de empezar a programar.

Es decir, **proceso de conversar y analizar juntos ya genera valor por sí solo**, independientemente de cuán preciso resulte el número final.

Por qué el esfuerzo de estimar debe ser bajo: la relación no lineal

Este es el punto más técnico y quizás el más importante del documento. La idea central:

"El remedio debe ser más barato que la enfermedad"

Esta frase resume la lógica económica detrás de esto: si **para estimar con más precisión hay que invertir tanto tiempo como el que llevaría directamente construir la funcionalidad, ya no tiene sentido** — el "costo de estimar" termina superando el valor que esa mayor precisión aporta.

Ejemplo: si estimar con precisión "perfecta" una historia requeriría 3 días de análisis detallado, pero la historia en sí se puede implementar en 2 días, claramente no vale la pena ese nivel de análisis previo. Es mejor aceptar una estimación aproximada, rápida, y ajustar sobre la marcha si hace falta.

Por eso se fija de antemano el tiempo destinado a estimar

Como consecuencia práctica, **en ágil se suele acotar de antemano cuánto tiempo se va a dedicar a estimar** (por ejemplo, un timebox de 1 hora para estimar todo el backlog priorizado de un sprint), evitando así que el equipo se quede "afinando" estimaciones indefinidamente en busca de una precisión que, de todas formas, no es alcanzable ni conviene perseguir.

Quién debería estimar cada tarea: el que la va a hacer

Se reafirma la idea de que **la persona (o el equipo) que va a ejecutar el trabajo es quien mejor puede estimarlo, porque conoce sus propias capacidades, ritmo y contexto**. Aun así, se insiste en mantener una **mente abierta** durante la discusión grupal — no se trata de imponer la propia visión, sino de contribuir con información real al proceso colectivo.

Por qué el Story Point resuelve el problema de comparar historias distintas

Cierre importante: como las historias suelen tener **dimensiones distintas** entre sí (una puede ser compleja, pero de poco esfuerzo, otra puede tener mucha incertidumbre técnica pero baja complejidad), no se pueden comparar "a ojo" sin una unidad común. **El Story Point actúa como esa unidad homogeneizadora, permitiendo comparar historias que, en su naturaleza, son bastante distintas entre sí — siempre considerando los tres elementos ya vistos: complejidad, esfuerzo e incertidumbre**

¿Qué se estima?

La respuesta es puntual: en ágil **solo se estima el tamaño** de las User Stories (o Features), usando Story Points, considerando los tres aspectos que ya vimos: **incertidumbre, complejidad y esfuerzo**.

Por qué se usa la serie de Fibonacci

La razón por la que se usan series como Fibonacci (1, 2, 3, 5, 8, 13, 21...) en vez de números consecutivos (1, 2, 3, 4, 5...) es que **la complejidad real de las historias tiende a crecer de forma exponencial, no lineal**.

No importa el valor puntual que se le asigna a una historia individual, importa que, dentro de un mismo equipo, esos valores sean **consistentes entre sí**.

Ejemplo clave del propio texto: una historia de "8 puntos" en el Equipo A puede no significar lo mismo que "8 puntos" en el Equipo B — no hay una equivalencia universal entre equipos. **Lo que sí tiene que cumplirse, siempre, es que, dentro del mismo equipo, una historia de 5 puntos sea consistentemente más chica que una de 8 puntos**. Los Story Points son una escala **interna** de cada equipo, no una unidad comparable entre distintos equipos.

Por qué no se estima en horas

Ya lo habíamos anticipado, pero acá se agrega una razón adicional muy concreta: la duración de los sprints es **fija** (el concepto de **Timebox**). Como el tiempo disponible ya está fijado de antemano, **no tiene sentido estimar "cuánto tiempo" va a llevar algo — lo que hace falta saber es cuánto entra dentro de ese tiempo ya fijo**. Por eso se estima tamaño (Story Points), no duración (horas).

Tamaño vs. Esfuerzo

Como el esfuerzo (horas de trabajo real) es el factor que más impacta en el **costo** de un proyecto, es natural que la gente tienda a pensar en términos de esfuerzo cuando en realidad debería pensar en términos de tamaño/complejidad.

Complejidad: independiente de quién hace el trabajo

La complejidad de una historia es una propiedad **intrínseca del problema**, no de la persona que lo resuelve. Un algoritmo complejo es complejo, sin importar si lo programa un desarrollador junior o senior.

Esfuerzo: depende de la persona

El esfuerzo, en cambio, **sí depende de quién ejecuta el trabajo**: su experiencia, capacitación, familiaridad con la tecnología. La misma historia puede requerir mucho esfuerzo de un desarrollador junior, y poco esfuerzo de uno senior con experiencia previa en ese tipo de problema — aunque la **complejidad** de la historia en sí sea exactamente la misma en ambos casos.

Horas lineales: un detalle práctico interesante

El texto agrega algo muy concreto sobre cómo pensar el esfuerzo en horas: de una jornada laboral de 8 horas, **no todas son horas efectivamente productivas**. Entre almuerzo, pausas, revisar emails, reuniones no relacionadas, etc., típicamente solo quedan **5.5 a 6 horas "lineales"** de trabajo real. Esto es importante tenerlo en cuenta si alguna vez se necesita traducir estimaciones a tiempo real de calendario (aunque, como vimos, no es lo que se busca estimar directamente en Story Points).

Esfuerzo ≠ calendario

El texto cierra con otra distinción clave: aunque se sepa cuántas horas de esfuerzo real requiere algo, **eso no garantiza una fecha exacta de entrega**, porque en el medio pueden pasar imprevistos (interrupciones, bloqueos, dependencias externas, imprevistos personales).

Velocidad

Se calcula, no se estima

Esta distinción de lenguaje es importante: la velocidad **no es una suposición ni una proyección**, es un **cálculo objetivo** basado en datos reales: la suma de Story Points de las historias que el equipo efectivamente completó en una iteración.

Qué cuenta como "completado"

Acá se retoma algo ya visto con el criterio "Small" de INVEST: **solo cuentan para la velocidad las historias que están 100% terminadas según la Definition of Done (desarrolladas, testeadas, y aceptadas por el Product Owner)**. Es, otra vez, **gestión binaria**: no existe el "casi completo".

Consecuencia directa: si una historia de 8 puntos quedó desarrollada, pero sin testear al final del sprint, esos 8 puntos **no cuentan** para la velocidad de esa iteración, aunque haya habido trabajo real invertido en ella.

Por qué importa el promedio, no la velocidad de una sola iteración

El equipo no debería obsesionarse con la velocidad de un sprint puntual, sino mirar el **promedio de velocidad a lo largo de varias iteraciones**. Ese promedio da una medida mucho más confiable y estable del ritmo real de trabajo del equipo, filtrando el ruido de variaciones puntuales.

Un equipo con suficiente historial de velocidad puede usar ese promedio para **estimar**, con razonable confianza, **cuántos Story Points podrá completar en el próximo sprint**, facilitando la planificación de releases y compromisos con el negocio.

Ejemplo: si un equipo tiene una velocidad promedio de 25 puntos por sprint en los últimos 6 sprints, puede planificar con relativa confianza que el próximo sprint también rondará esa cifra, y tomar del backlog historias que sumen aproximadamente 25 puntos — sabiendo que, si bien puede haber variación puntual, el promedio histórico es un buen predictor.

Reestimación

Este texto es breve, pero aporta una distinción importante: **cuándo sí conviene reestimar una User Story, y cuándo no**, un tema donde suele haber confusión en equipos que recién empiezan con ágil.

Cuando Sí conviene reestimar

La única razón legítima para reestimar una historia ya estimada es cuando **cambia la percepción relativa de su tamaño**, en comparación con otras historias.

Esto suele pasar como efecto colateral natural del propio proceso de estimación continuo: a **medida que el equipo sigue estimando nuevas historias, puede descubrir que algunas de las ya estimadas anteriormente en realidad son más grandes o más chicas de lo que se pensó en su momento**, en relación a estas nuevas referencias.

Planificación en Ambientes Ágiles

Los 3 niveles de planificación

Un proyecto necesita una visión de mediano plazo (release), un compromiso de corto plazo (iteración), y una coordinación de trabajo diaria — cada uno con su propio ritmo y propósito.

1. Planificación de Release (la más "macro")

- **Cuando:** al inicio del proyecto, y se actualiza periódicamente (normalmente al comienzo de cada iteración).
- **Qué define:** qué historias, épicas o temas entran en una entrega del producto.
- **Cómo se calcula la duración:** se priorizan las historias, se estiman en Story Points y se divide el total por la velocidad promedio del equipo.
- **Ejemplo:** 150 puntos ÷ 25 puntos por sprint = unos 6 sprints. Es una proyección, no una promesa (estimación ≠ compromiso).

2. Planificación de Iteración (Sprint Planning)

- **Cuando:** al comienzo de cada sprint.
- **Quién la lidera:** el Product Owner, que prioriza las historias según lo aprendido en el sprint anterior.
- **Qué queda fijo y qué se ajusta:** el tiempo (timebox) y la velocidad son fijos. Lo que se ajusta es el alcance, es decir, cuántas historias entran.
- **Qué se discute:** qué historias entran y cómo se convierten en tareas técnicas concretas.
- **Ejemplo:** para la historia "recuperar contraseña por email", las tareas serían el formulario, el endpoint, el envío de emails y las pruebas de aceptación.

3. Planificación del Día (Daily Standup)

- **Cuándo:** todos los días, en una reunión breve.
- **Qué se discute:** solo las tareas concretas del día para avanzar con una historia.
- **Para qué sirve:** coordinar al equipo y detectar bloqueos rápido, sin esperar a la próxima planificación.
- **Ejemplo:** "Ayer terminé el endpoint, hoy integro el envío de emails y estoy bloqueado por las credenciales del servicio". Así el equipo o el Scrum Master pueden destrabarlo enseguida.